BÁO CÁO TIẾN ĐỘ HỌC TẬP

# Linux, Virtualization

# → Container & Kubernetes

Tổng kết kiến thức Tuần 1 và Tuần 2

Linux · Network · Storage · Permission · Hypervisor · Container · Kubernetes

Người báo cáo: **Lê Tấn Lực** · **Đỗ Hoàng Hiếu**

TỔNG QUAN

# Tiến độ 2 tuần đầu

| 2 | 8 | 6 | 2 |
|---|---|---|---|
| tuần đã hoàn thành | mảng kiến thức | mảng hạ tầng nền tảng | mảng Container & K8s |

## Tuần 1 – Linux, Virtualization và hạ tầng nền tảng

1. Linux doanh nghiệp: CPU, RAM, disk, process
2. Network: IP, routing, DNS, kết nối VM
3. Storage: partition, LVM, filesystem, mount
4. User, group và permission
5. Service (systemd) và firewall
6. Hypervisor và vòng đời VM

## Tuần 2 – Container và Kubernetes Foundation

7. Container: image, registry, volume, network, resource limit
8. Kiến trúc Kubernetes và vai trò từng thành phần

**Mạch kiến thức:** hiểu máy chủ Linux → ảo hóa thành nhiều VM → đóng gói ứng dụng thành container → điều phối container ở quy mô cluster bằng Kubernetes.

TUẦN 1

# Linux, Virtualization và hạ tầng nền tảng

Tài nguyên máy chủ | Network | Storage & LVM | User & Permission | systemd & Firewall | Hypervisor & VM

TUẦN 1 · TỔNG QUAN

# Mục tiêu Tuần 1 và tiêu chí đầu ra

| # | Chủ đề | Tiêu chí đầu ra (theo roadmap) |
|---|---|---|
| 1 | Linux doanh nghiệp: CPU, RAM, disk, process | Kiểm tra và phân bổ được tài nguyên; giải thích được cho người khác |
| 2 | Network: IP, routing, DNS, kết nối VM | Vẽ và giải thích được luồng kết nối mạng giữa các VM trong Lab |
| 3 | Storage: partition, LVM, filesystem, mount | Tự thêm được disk mới, tạo LVM và mount vào hệ thống |
| 4 | User, group và permission | Tạo và phân quyền được theo đúng yêu cầu của ứng dụng |
| 5 | Service (systemd) và firewall | Quản lý được vòng đời dịch vụ; mở/đóng port theo yêu cầu |
| 6 | Hypervisor và vòng đời VM | Thao tác đầy đủ vòng đời VM trên hypervisor đang dùng |

**Cách tiếp cận:** đo → xác định nút thắt (bottleneck) → phân bổ hoặc giới hạn.

**Môi trường Lab:** 2 VM `vm-web` (nginx) và `vm-db` (mariadb) trên KVM / VirtualBox.

**Mỗi chủ đề gồm:** lý thuyết, lệnh thực hành và một ví dụ dễ hiểu để trình bày.

# CPU và Load Average

## Cấu trúc CPU

- **Socket → Core → Thread:** Hyper-Threading cho mỗi core 2 thread. Linux coi mỗi thread là 1 **CPU logic** ( `lscpu` , `nproc` )
- **CPU time:** `us` ứng dụng `sy` kernel `ni` process đã nice `id` rảnh `wa` chờ disk `st` hypervisor lấy cho VM khác

## Load average (1 · 5 · 15 phút)

Số process trung bình **đang chạy hoặc chờ CPU** cộng với số process đang chờ I/O (trạng thái `D` ). Đem so với số CPU logic:

- Load **4.0** trên máy 4 CPU = đầy tải 100%
- Load **8.0** trên máy 4 CPU = hàng đợi gấp đôi

```
lscpu | grep -E "^CPU\(s\)|Thread|Core|Socket"
uptime              # load average: 0.52, 0.40, 0.31
top                 # nhấn 1: từng CPU · P: sắp theo CPU
mpstat -P ALL 1 3   # thống kê từng core (sysstat)
vmstat 1 5          # r = hàng đợi CPU, wa = chờ I/O
```

| Hiện tượng | Ý nghĩa | Hướng xử lý |
|---|---|---|
| `us` cao | Ứng dụng tính toán nặng | Tối ưu code, thêm vCPU |
| `sy` cao | Nhiều system call | `pidstat -w` , xem số thread |
| `wa` cao | CPU rảnh, chờ disk | Điều tra disk, không phải thiếu CPU |
| `st` cao | Host bị overcommit | Báo đội hạ tầng, migrate VM |

# Bộ nhớ (RAM, swap) và Disk

```
free -h
#               total        used        free      buff/cache   available
# Mem:           3.8Gi       1.1Gi       210Mi       2.5Gi       2.4Gi
vmstat 1    # si/so > 0 liên tục = đang swap → thiếu RAM
```

## RAM

- Linux dùng RAM rảnh làm **page cache**, nên *free* thấp là bình thường. Chỉ số cần nhìn là **available**
- **RSS** = RAM vật lý đang dùng · **VSZ** = không gian ảo (thường lớn, không đáng lo)
- **Swap** chậm hơn RAM hàng nghìn lần. Với DB server nên đặt `vm.swappiness` = 1–10
- Hết RAM và swap → **OOM Killer** giết process ( `dmesg -T | grep -i oom` )

## Disk: hai câu hỏi khác nhau

**Còn bao nhiêu chỗ?** (capacity) và **disk có đang nghẽn không?** (I/O)

```
df -hT ; df -i          # hết inode cũng "đầy disk"
du -sh /var/* | sort -h # thư mục chiếm chỗ
iostat -xz 1 3          # %util ~100%, await cao = nghẽn
sudo iotop -o           # process nào đang đọc/ghi
lsof +L1                # file đã xoá nhưng còn bị giữ
```

| Chỉ số iostat | Ý nghĩa |
|---|---|
| `r/s` , `w/s` | Số IOPS đọc/ghi mỗi giây |
| `r_await` , `w_await` | Độ trễ (ms). SSD nên dưới vài ms |
| `%util` | ~100% liên tục = disk bão hòa |

# Process, giới hạn tài nguyên và phương pháp USE

## Process

- Trạng thái: `R` chạy · `S` ngủ · `D` chờ I/O · `T` dừng · `Z` zombie
- Signal: **SIGTERM 15** dừng lịch sự · **SIGKILL 9** buộc dừng · **SIGHUP 1** reload cấu hình
- `nice` từ −20 (ưu tiên cao) đến 19. Giảm nice cần quyền root
- **cgroups** giới hạn CPU/RAM/IO cho nhóm process. Đây là nền tảng của systemd và container

```
ps -eo pid,ppid,user,stat,%cpu,%mem,cmd --sort=-%cpu | head
kill -15 1234 ; pkill -HUP nginx
nice -n 10 tar czf backup.tgz /data
ionice -c3 -p 1234              # I/O ưu tiên "idle"
sudo systemctl set-property nginx.service \
     CPUQuota=50% MemoryMax=512M
```

**USE Method:** với mỗi tài nguyên, kiểm tra **U**tilization (mức dùng), **S**aturation (hàng đợi) và **E**rrors (lỗi). Quy trình 60 giây: `uptime` → `dmesg -T | tail` → `vmstat` → `mpstat` → `pidstat` → `iostat` → `free` → `sar -n DEV`

💡 **Ví dụ trình bày: server như một nhà hàng**

**CPU** là đầu bếp, **load** là số món đang chờ nấu: 4 đầu bếp mà 8 món chờ là quá tải gấp đôi. **RAM** là mặt bàn bếp, **cache** là nguyên liệu bày sẵn, dọn đi được ngay. **%wa** cao là đầu bếp đứng chờ người lấy đồ trong kho (disk).

🎯 **Đầu ra:** Kiểm tra và phân bổ được tài nguyên; giải thích được cho người khác

# Địa chỉ IP, CIDR và routing

| CIDR | Mask | Host dùng được | Ví dụ |
|---|---|---|---|
| /8 | 255.0.0.0 | 16.777.214 | 10.0.0.0/8 (private) |
| /16 | 255.255.0.0 | 65.534 | 172.16.0.0/12 |
| /24 | 255.255.255.0 | 254 | 192.168.56.0/24 |
| /28 | 255.255.255.240 | 14 | Subnet nhỏ cho DMZ |
| /32 | 255.255.255.255 | 1 | Route tới 1 IP |

- **Số host = 2^(32 − prefix) − 2** (trừ địa chỉ network và broadcast)
- `192.168.56.10/24` → network `.0`, broadcast `.255`, host từ `.1` đến `.254`
- **Private (RFC 1918)** không ra Internet được, phải đi qua NAT
- Cùng subnet thì gửi thẳng qua switch (ARP lấy MAC). Khác subnet thì phải qua gateway

## Gói tin đi đâu?

- Kernel chọn route **khớp dài nhất** (longest prefix match). Không khớp route nào thì đi theo **default route** `0.0.0.0/0`
- Có nhiều route cùng độ dài thì route có **metric** thấp hơn được ưu tiên
- **Lỗi hay gặp:** 2 interface cùng có default gateway → traffic ra sai cổng. Chỉ nên có 1 default route

```
ip -br addr                 # interface + IP gọn
ip route
# default via 10.0.2.2 dev enp0s3 metric 100
# 192.168.56.0/24 dev enp0s8 src 192.168.56.10
ip route get 8.8.8.8        # kernel chọn route nào
ip route add 10.10.0.0/16 via 192.168.56.1
sysctl -w net.ipv4.ip_forward=1   # Linux làm router
```

# Cấu hình IP vĩnh viễn và DNS

```
# RHEL/Rocky - NetworkManager
nmcli con mod enp0s8 ipv4.method manual \
  ipv4.addresses 192.168.56.10/24 ipv4.gateway 192.168.56.1 \
  ipv4.dns "8.8.8.8 1.1.1.1"
nmcli con up enp0s8
```

```
# Ubuntu - Netplan: /etc/netplan/01-lab.yaml
network:
  version: 2
  ethernets:
    enp0s8:
      addresses: [192.168.56.10/24]
      nameservers: { addresses: [8.8.8.8] }
# netplan try  → tự rollback sau 120s nếu mất kết nối
```

Lệnh `ip addr add` chỉ có tác dụng tạm thời và mất khi reboot. Muốn cấu hình vĩnh viễn phải dùng nmcli hoặc Netplan.

## DNS phân giải tên

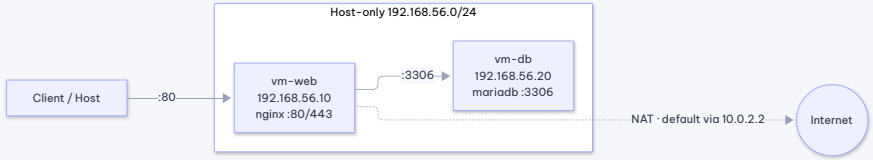
- Thứ tự phân giải ghi trong `/etc/nsswitch.conf` ( `hosts: files dns` ): đọc `/etc/hosts` trước, sau đó mới hỏi DNS server trong `/etc/resolv.conf`
- **Luồng truy vấn:** client → resolver (8.8.8.8) → root → TLD ( `.com` ) → authoritative. Kết quả được cache theo **TTL**
- **Bản ghi:** A · AAAA · CNAME · MX · NS · PTR · TXT
- Ubuntu dùng **systemd-resolved**: resolv.conf trỏ tới `127.0.0.53`, xem cấu hình thật bằng `resolvectl status`

```
dig google.com +short ; dig -x 8.8.8.8
dig @1.1.1.1 example.com MX
getent hosts vm-db       # đi qua nsswitch (cả /etc/hosts)
# /etc/hosts trong lab
192.168.56.10  vm-web
192.168.56.20  vm-db
```

# Mạng ảo cho VM và sơ đồ Lab

| Chế độ | Cách hoạt động | Internet | Host↔VM | VM↔VM |
|---|---|---|---|---|
| **NAT** | Hypervisor làm router NAT | ✅ | Port fwd | Tùy |
| **Bridged** | Như máy thật trong LAN | ✅ | ✅ | ✅ |
| **Host-only** | Mạng riêng host + VM | ❌ | ✅ | ✅ |
| **Internal** | Chỉ VM với VM | ❌ | ❌ | ✅ |

**KVM/libvirt:** `virbr0` (NAT 192.168.122.0/24, DHCP dnsmasq), bridge `br0` cho bridged · **VMware:** VMnet8 NAT, VMnet1 Host-only, VMnet0 Bridged · **Proxmox:** Linux bridge `vmbr0`




**Luồng:** Client → `vm-web:80` qua mạng host-only → `vm-db:3306`. Riêng `vm-web` ra Internet qua NAT (NIC `enp0s3`, 10.0.2.15).

💡 **Ví dụ trình bày: mạng như hệ thống bưu điện**

**IP** là địa chỉ nhà, **subnet /24** là cùng một con phố nên gửi thư thì đưa tận tay (switch/ARP). **Gateway** là bưu cục đầu phố, **routing table** là bảng chỉ đường chọn tuyến cụ thể nhất. **DNS** là danh bạ, tra tên "vm-db" ra 192.168.56.20 trước khi gửi.

🎯 **Đầu ra:** Vẽ và giải thích được luồng kết nối mạng giữa các VM trong Lab

# Chuỗi lưu trữ, partition và LVM




| | MBR (msdos) | GPT |
|---|---|---|
| Disk tối đa | 2 TiB | ~9.4 ZB (gần như không giới hạn) |
| Số partition | 4 primary (hoặc 3 + extended) | 128 mặc định |
| Boot | BIOS legacy | UEFI |
| Độ bền | 1 bảng duy nhất | Có bản sao header ở cuối disk + CRC |

```
lsblk -f ; blkid ; findmnt       # xem thiết bị, UUID, mount
parted /dev/sdb --script mklabel gpt \
  mkpart primary 0% 100% set 1 lvm on
```

| Thành phần | Là gì | Lệnh |
|---|---|---|
| **PV** Physical Volume | Disk/partition được khởi tạo cho LVM | `pvcreate`, `pvs` |
| **VG** Volume Group | "Bể" dung lượng gộp từ nhiều PV, chia thành PE (4 MiB) | `vgcreate`, `vgextend` |
| **LV** Logical Volume | "Partition ảo" cắt từ VG, nơi tạo filesystem | `lvcreate`, `lvextend` |

**Vì sao dùng LVM:** có một lớp trừu tượng giữa disk và filesystem, nhờ đó gộp được nhiều disk, **mở rộng khi đang chạy** và tạo snapshot.

# Thực hành LVM, filesystem và /etc/fstab

```
pvcreate /dev/sdb1
vgcreate vg_data /dev/sdb1
lvcreate -n lv_app -L 5G vg_data      # hoặc -l 100%FREE
mkfs.xfs /dev/vg_data/lv_app
# Mở rộng khi thêm disk /dev/sdc – không downtime
pvcreate /dev/sdc && vgextend vg_data /dev/sdc
lvextend -r -L +3G /dev/vg_data/lv_app   # -r: resize FS
# Snapshot LV trước khi nâng cấp
lvcreate -s -n snap_app -L 1G /dev/vg_data/lv_app
```

```
echo 'UUID=<uuid> /data/app xfs defaults,nofail 0 0' >> /etc/fstab
mount -a && findmnt --verify     # LUÔN kiểm tra trước reboot
```

| Tiêu chí | ext4 | xfs |
|---|---|---|
| Mặc định trên | Ubuntu, Debian | RHEL, Rocky, Alma |
| Mở rộng online | `resize2fs` | `xfs_growfs` |
| Thu nhỏ | Được (phải umount) | **Không hỗ trợ** |
| Điểm mạnh | Ổn định, linh hoạt | File lớn, song song cao |

### 6 cột của fstab

thiết bị · mount point · loại FS · options ( `defaults` , `noatime` , `nofail` , `ro` ) · dump · fsck pass (0 tắt, 1 cho `/` , 2 cho FS khác)

- Dùng **UUID** vì tên `/dev/sdX` có thể đổi khi thêm disk
- Một dòng sai có thể khiến máy vào **emergency mode**. `nofail` tránh lỗi này với disk phụ

**Ví dụ trình bày: storage như kho hàng**

**Disk** là mảnh đất, **partition** là chia lô, **LVM** gộp nhiều lô thành một khu (VG) rồi chia lại linh hoạt thành các kho (LV). **Filesystem** là cách xếp kệ, **mount** là gắn biển địa chỉ "/data/app". Sắp đầy thì thêm disk, chạy vgextend + lvextend -r, không downtime.

**Đầu ra:** Tự thêm được disk mới, tạo LVM và mount vào hệ thống

# User, group và quyền cơ bản rwx

| File | Nội dung |
|---|---|
| `/etc/passwd` | user:x:UID:GID:mô tả:home:shell |
| `/etc/shadow` | Mật khẩu đã hash, chính sách hết hạn (chỉ root đọc) |
| `/etc/group` | group:x:GID:danh sách thành viên |
| `/etc/sudoers.d/` | Quy tắc sudo, sửa bằng `visudo` |

```
useradd -m -s /bin/bash -c "Dev Luc" luc ; passwd luc
groupadd devteam
usermod -aG devteam luc     # thiếu -a sẽ XOÁ các group cũ!
useradd -r -s /usr/sbin/nologin -d /opt/myapp myapp
chage -M 90 luc ; usermod -L luc ; userdel -r olduser
```

UID 0 là root. UID dưới 1000 là system user dành cho dịch vụ (nginx, mysql), không có shell đăng nhập.

| Quyền | Số | Với file | Với thư mục |
|---|---|---|---|
| r | 4 | Đọc nội dung | Liệt kê tên file ( `ls` ) |
| w | 2 | Sửa nội dung | Tạo / xoá / đổi tên file bên trong |
| x | 1 | Thực thi | Đi vào ( `cd` ), truy cập file bên trong |

### 3 quy tắc hay bị nhầm

1. Kernel kiểm tra theo thứ tự **owner** → **group** → **others**, áp dụng nhóm đầu tiên khớp, **không cộng dồn**
2. Đọc `/a/b/file` cần quyền `x` trên `/a` và `/a/b`. Kiểm tra bằng `namei -l`
3. **Xoá file** phụ thuộc quyền `w` của **thư mục**, không phụ thuộc quyền của file

```
chmod 640 config.yml ; chmod -R g+rX /srv/share
chown -R myapp:devteam /opt/myapp
umask      # 0022 → file mới 644, thư mục mới 755
```

# Quyền đặc biệt, ACL và sudo

| Bit | Số | Ý nghĩa | Ví dụ |
|---|---|---|---|
| **SUID** | 4000 | File chạy với quyền của owner | `/usr/bin/passwd` |
| **SGID** | 2000 | File mới trong thư mục kế thừa group | Thư mục dự án chung |
| **Sticky** | 1000 | Chỉ owner mới xoá được file của mình | `/tmp` (drwxrwxrwt) |

```
chmod 2775 /srv/project                  # SGID thư mục nhóm
find / -perm -4000 -type f 2>/dev/null   # audit file SUID
setfacl -m u:deploy:rx /opt/myapp        # ACL cho 1 user
setfacl -d -m g:devteam:rwX /srv/project # default ACL
getfacl /opt/myapp
```

```
# visudo -f /etc/sudoers.d/devteam
%devteam ALL=(root) /usr/bin/systemctl restart myapp, \
                    /usr/bin/journalctl -u myapp
deploy   ALL=(myapp) NOPASSWD: /opt/myapp/bin/deploy.sh
sudo -l -U luc        # xem user được phép chạy gì
```

### Nguyên tắc quyền tối thiểu

- Không đăng nhập trực tiếp bằng root. Tắt `PermitRootLogin` trong sshd_config
- Mỗi ứng dụng một system user riêng, không có shell, chỉ sở hữu thư mục của nó
- **SELinux** (RHEL): rwx đúng mà vẫn bị từ chối thì kiểm tra `ls -Z`, `ausearch -m avc`, `restorecon -Rv`

💡 **Ví dụ trình bày: phân quyền như thẻ ra vào toà nhà**

**User** là nhân viên, **group** là phòng ban, **rwx** là được xem, sửa, đi vào. Chỉ user `myapp` đọc được file mật khẩu DB (600), dev chỉ đọc log, deploy ghi code qua ACL. **sudo giới hạn** là thẻ chỉ mở đúng phòng cần thiết, không phải chìa khoá vạn năng.

🎯 **Đầu ra:** Tạo và phân quyền được theo đúng yêu cầu của ứng dụng

# systemd và vòng đời dịch vụ

**systemd** là PID 1 trên hầu hết bản Linux hiện đại. Nó khởi động hệ thống, quản lý dịch vụ, log (journald), mount và timer. Đơn vị quản lý là **unit**: `.service` `.socket` `.timer` `.mount` `.target`

| Lệnh `systemctl …` | Tác dụng |
|---|---|
| `start` / `stop` / `restart` | Chạy / dừng / khởi động lại ngay |
| `reload` | Đọc lại cấu hình, không ngắt kết nối |
| `enable --now` | Tự chạy khi boot và chạy ngay |
| `disable` / `mask` | Không tự chạy / cấm hoàn toàn |
| `status` · `list-units --failed` | Trạng thái, PID, log gần nhất · dịch vụ lỗi |
| `daemon-reload` | **Bắt buộc** sau khi sửa unit file |

```
systemctl edit nginx       # tạo override thay vì sửa file gốc
systemd-analyze blame      # dịch vụ nào làm boot chậm
journalctl -u myapp -f ; journalctl -b -1   # log boot trước
```

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=myapp
WorkingDirectory=/opt/myapp
EnvironmentFile=/etc/myapp/app.env
ExecStart=/opt/myapp/bin/server --port 8080
Restart=on-failure
RestartSec=5
MemoryMax=512M
CPUQuota=50%
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
```

# Firewall và kiểm tra port từ đầu đến cuối

```
# firewalld (RHEL/Rocky) – quản lý theo zone
firewall-cmd --get-active-zones ; firewall-cmd --list-all
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --permanent --add-rich-rule='rule family=ipv4
  source address=192.168.56.10/32 port port=3306
  protocol=tcp accept'      # chỉ vm-web được vào DB
firewall-cmd --reload       # permanent có hiệu lực sau reload
```

```
# ufw (Ubuntu)
ufw default deny incoming ; ufw default allow outgoing
ufw allow OpenSSH           # LUÔN mở SSH trước khi enable!
ufw allow 80/tcp
ufw allow from 192.168.56.10 to any port 3306
ufw enable ; ufw status numbered ; ufw delete 3
# mức thấp: nft list ruleset ; iptables -L -n -v
```

## Kiểm tra port end-to-end

`ss -tulpn` (dịch vụ có LISTEN không, trên 0.0.0.0 hay 127.0.0.1?) → `nc -zv 192.168.56.20 3306` từ máy khác → `curl -I http://192.168.56.10`

- Dịch vụ chỉ listen **127.0.0.1** thì máy khác không vào được dù firewall đã mở. Phải sửa `bind-address`
- RHEL dùng port lạ thì cần mở cả SELinux: `semanage port -a -t http_port_t -p tcp 8081`

💡 **Ví dụ trình bày: systemd là quản lý ca, firewall là bảo vệ cổng**

systemd cho dịch vụ vào ca khi boot (**enable**), tự gọi lại khi dịch vụ "ngã" (**Restart=on-failure**) và ghi sổ nhật ký (**journalctl**). Firewall mặc định đóng mọi cửa, chỉ mở 80/443 cho khách. Cửa 3306 của DB chỉ mở cho đúng vm-web, nhờ vậy bề mặt tấn công nhỏ nhất.

🎯 **Đầu ra:** Quản lý được vòng đời dịch vụ; mở/đóng port theo yêu cầu

# Hypervisor là gì?

**Hypervisor (VMM)** chia tài nguyên vật lý thành nhiều máy ảo độc lập. CPU hiện đại hỗ trợ ảo hóa phần cứng (**Intel VT-x / AMD-V**, kiểm tra bằng `grep -cE "vmx|svm" /proc/cpuinfo`) để VM chạy gần tốc độ thật.

| | Type 1 (bare-metal) | Type 2 (hosted) |
|---|---|---|
| Chạy trên | Trực tiếp phần cứng | Một OS có sẵn |
| Ví dụ | ESXi, Proxmox VE (KVM), Hyper-V, Xen, Nutanix AHV | VirtualBox, VMware Workstation/Fusion |
| Dùng cho | Production, datacenter | Lab cá nhân, dev |

### KVM · QEMU · libvirt

- **KVM**: module kernel biến Linux thành hypervisor
- **QEMU**: giả lập thiết bị cho VM
- **libvirt**: lớp quản lý (`virsh`, `virt-install`, `virt-manager`)
- Proxmox và OpenStack đều dựa trên KVM

### Thiết bị paravirtual

`virtio-blk/scsi` cho disk, `virtio-net` cho NIC, hiệu năng tốt hơn thiết bị giả lập. VMware dùng VMware Tools, `vmxnet3`, `pvscsi`.

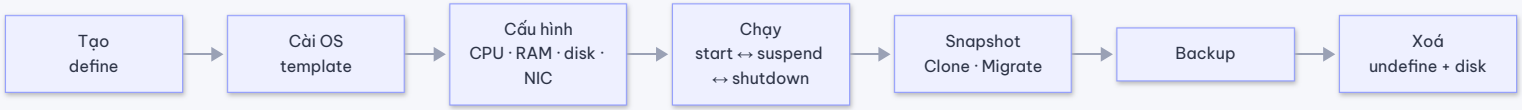
### Overcommit

Cấp tổng vCPU/RAM cho các VM vượt tài nguyên thật. **vCPU 3–4:1** thường chấp nhận được. **RAM overcommit rủi ro hơn** (balloon, host phải swap).

### Định dạng disk

**qcow2** (KVM, thin + snapshot) · **raw** (nhanh nhất) · **vmdk** (VMware) · **vhdx** (Hyper-V). Thin provisioning tiết kiệm chỗ nhưng có thể làm đầy datastore.

# Vòng đời VM và thao tác trên các nền tảng




```
virt-install --name vm-web --memory 2048 --vcpus 2 \
  --disk size=20,format=qcow2,bus=virtio \
  --cdrom /iso/Rocky-9.iso --os-variant rocky9 \
  --network network=default,model=virtio
virsh list --all ; virsh dominfo vm-web
virsh start | shutdown | suspend | resume vm-web
virsh setvcpus vm-web 4 --config ; virsh setmem vm-web 4G --config
virsh attach-disk vm-web data.qcow2 vdb --persistent
virt-clone --original vm-web --name vm-web2 --auto-clone
virsh undefine vm-web2 --remove-all-storage
```

| Thao tác | Proxmox ( `qm` ) | VMware ESXi | VirtualBox |
|---|---|---|---|
| Tạo | `qm create 101 --memory 2048 --cores 2` | Wizard / PowerCLI `New-VM` | `createvm` + `modifyvm` |
| Start / Stop | `qm start` / `shutdown 101` | Power On / Shut Down | `startvm --type headless` |
| Snapshot | `qm snapshot 101 pre-upd` | Take Snapshot | `snapshot vm take` |
| Backup | `vzdump 101 --mode snapshot` | Veeam, vSphere Data Protection | `export vm -o vm.ova` |

# Snapshot và Backup không phải là một

| | Snapshot | Backup |
|---|---|---|
| Bản chất | Điểm khôi phục, lưu phần chênh lệch (delta) so với disk gốc | Bản sao độc lập, đầy đủ của dữ liệu |
| Vị trí | Cùng storage với VM | Storage khác, tốt nhất ở site khác |
| Mất storage gốc | **Mất luôn** snapshot | **Vẫn khôi phục được** |
| Dùng khi | Trước khi update, rollback nhanh | Phòng thảm họa, lưu trữ lâu dài |
| Giữ lâu | Không nên (làm chậm I/O, phình disk) | Theo chính sách retention |

```
virsh snapshot-create-as vm-web pre-update "Truoc khi update kernel"
virsh snapshot-revert vm-web pre-update
virsh dumpxml vm-web > /backup/vm-web.xml      # backup cold
qemu-img convert -O qcow2 -c vm-web.qcow2 /backup/vm-web.qcow2
```

**Quy tắc 3-2-1:** 3 bản dữ liệu, trên 2 loại phương tiện, 1 bản ở ngoài site.

**Application-consistent:** với DB, dùng qemu-guest-agent ( `virsh domfsfreeze` ) hoặc dump DB trước khi snapshot để tránh dữ liệu ghi dở.

**RPO / RTO:** mất tối đa bao nhiêu dữ liệu / khôi phục trong bao lâu. Backup chưa từng restore thử thì coi như chưa có backup.

💡 **Ví dụ: hypervisor như toà chung cư**

Hypervisor là ban quản lý chia căn hộ (VM). **Snapshot** là chụp ảnh căn hộ trước khi sửa, nhưng ảnh vẫn nằm trong toà nhà. **Backup** là bản sao cất ở toà nhà khác, cháy toà nhà vẫn khôi phục được.

🎯 **Đầu ra:** Thao tác đầy đủ vòng đời VM trên hypervisor đang dùng

TUẦN 1 · ÁP DỤNG

# Troubleshooting nhanh

| Triệu chứng | Kiểm tra | Hướng xử lý |
|---|---|---|
| Server chậm | `uptime`, `top`, `vmstat`, `iostat` | Xác định CPU / RAM / IO, xử lý process nguyên nhân |
| Boot vào emergency mode | `journalctl -xb`, `/etc/fstab` | Sửa dòng fstab sai, thêm `nofail` |
| Disk đầy | `df -h`, `df -i`, `du`, `lsof +L1` | Dọn log (logrotate), restart process giữ file, mở rộng LV |
| Service không chạy | `systemctl status`, `journalctl -u` | Sửa config hoặc quyền, `daemon-reload` |
| Không truy cập được port | `ss -tulpn`, `firewall-cmd --list-all`, `nc -zv` | Sửa bind address, mở firewall, SELinux port |
| Permission denied | `id`, `namei -l`, `getfacl`, `ls -Z` | Sửa owner / mode / ACL, `restorecon` |
| VM không có mạng | `ip a`, `ip r`, `virsh domiflist`, `virsh net-list` | Kiểm tra NIC model, bridge, DHCP, default route |

**Tài liệu tham khảo:** RHEL Documentation · Ubuntu Server Guide · Brendan Gregg – USE Method · *The Linux Command Line*, *How Linux Works* · libvirt.org, Proxmox VE Admin Guide. Chứng chỉ phù hợp: **RHCSA (EX200)**, **LFCS**.

TUẦN 2

# Container và Kubernetes Foundation

Image & Registry | Volume | Network | Resource limit | Kiến trúc Kubernetes

TUẦN 2 · CHỦ ĐỀ 7

# Container hoạt động như thế nào?

### Container là tiến trình được cô lập

Container không phải máy ảo. Đó là tiến trình chạy trên **kernel của host**, được kernel Linux cô lập bằng 3 cơ chế:

- **Namespaces**: cô lập những gì tiến trình *nhìn thấy*: `pid`, `net`, `mnt`, `uts` (hostname), `ipc`, `user`
- **cgroups**: giới hạn tài nguyên tiến trình *được dùng*: CPU, RAM, I/O
- **Union filesystem** (overlay2): ghép các layer chỉ đọc của image với một layer ghi riêng cho container

| Tiêu chí | Virtual Machine | Container |
|---|---|---|
| Ảo hóa | Phần cứng (hypervisor) | Hệ điều hành (kernel) |
| Hệ điều hành | Mỗi VM một Guest OS | Dùng chung kernel host |
| Kích thước | Vài GB | Vài chục – vài trăm MB |
| Khởi động | Phút | Giây |
| Mức cô lập | Mạnh | Yếu hơn |
| Phù hợp | Nhiều OS, cô lập cao | Microservice, CI/CD |

**Tóm lại:** VM ảo hóa cả một máy tính. Container chỉ cô lập ứng dụng, nên nhẹ và nhanh hơn nhưng phải dùng chung kernel.

🎯 **Đầu ra:** Giải thích được cơ chế container và khác biệt so với VM trong 5 phút

TUẦN 2 · CHỦ ĐỀ 7

# Image và Registry

## Image

- Template **chỉ đọc**, gồm nhiều **layer**. Mỗi lệnh trong Dockerfile tạo một layer
- Layer được **cache và dùng chung** giữa các image → build và pull nhanh
- Container = image + một layer ghi. Xóa container thì layer ghi mất

## Registry

- Nơi lưu và phân phối image: Docker Hub, Harbor, GitLab Registry, ECR...
- Tên đầy đủ: `registry.company.vn/team/webapp:1.4.2`
- **Không dùng tag** `latest` cho production vì không biết chính xác phiên bản đang chạy

```
# Multi-stage build: image cuối nhỏ gọn
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci          # layer được cache
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
USER node           # không chạy bằng root
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

```
docker build -t registry.company.vn/team/webapp:1.4.2 .
docker push registry.company.vn/team/webapp:1.4.2
```

# Volume, network và resource limit

## Volume

Dữ liệu trong container mất khi container bị xóa, nên dữ liệu cần giữ phải để ngoài:

- **Named volume**: Docker quản lý, dùng cho database
- **Bind mount**: gắn thư mục host, tiện khi dev
- **tmpfs**: nằm trên RAM, dữ liệu tạm

## Network

- **bridge** (mặc định): mạng riêng trên host, ra ngoài qua NAT
- **user-defined bridge**: container gọi nhau **bằng tên** nhờ DNS nội bộ
- **host**: dùng chung mạng của host
- **none**: không có mạng
- `-p 8080:80` : publish port host 8080 → container 80

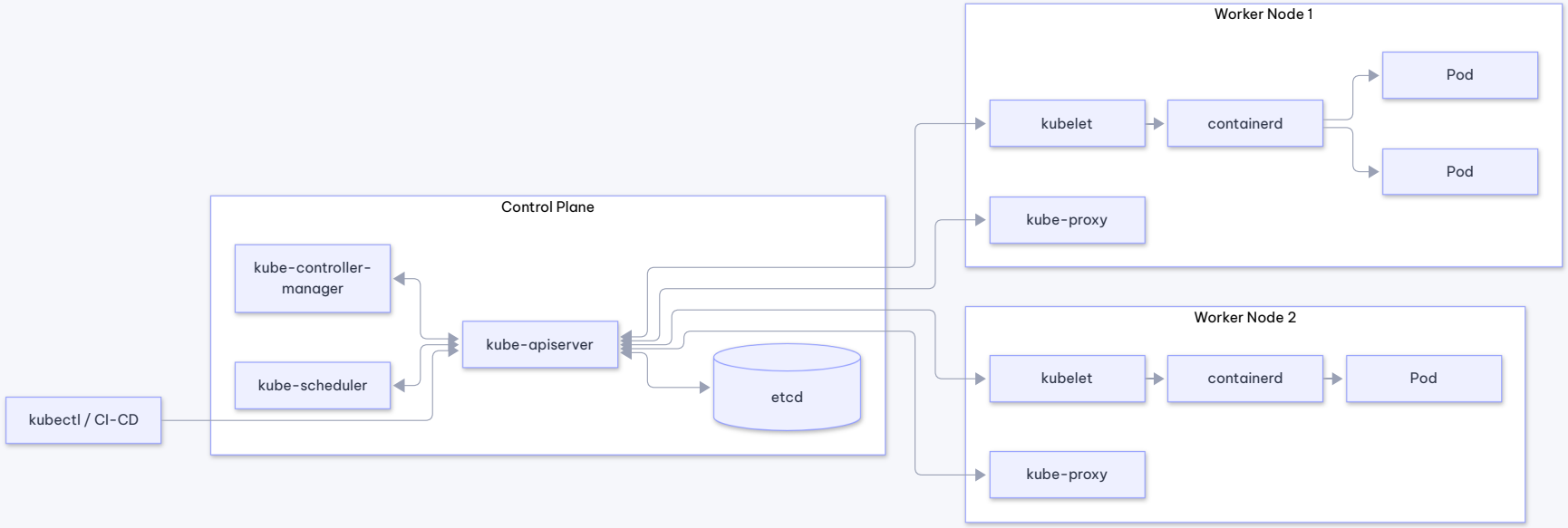
## Resource limit

- `--memory 512m` : vượt giới hạn thì container bị **OOM kill**
- `--cpus 1.5` : tối đa 1,5 core
- `--pids-limit` : giới hạn số tiến trình
- Không đặt limit thì một container có thể chiếm hết tài nguyên của host

```
docker network create appnet
docker volume create pgdata
docker run -d --name db  --network appnet -v pgdata:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD_FILE=/run/secrets/db_pass --memory 1g --cpus 1 postgres:16
docker run -d --name web --network appnet -p 8080:3000 \
  -e DB_HOST=db --memory 512m --cpus 0.5 registry.company.vn/team/webapp:1.4.2
docker stats                    # theo dõi CPU/RAM thực tế của từng container
```

TUẦN 2 · CHỦ ĐỀ 8

# Kiến trúc Kubernetes




**Control Plane** là "bộ não": nhận yêu cầu, lưu trạng thái mong muốn, quyết định Pod chạy ở đâu và liên tục sửa lệch.

**Worker Node** là nơi chạy workload: nhận lệnh từ apiserver, chạy container và cấu hình mạng cho Service.

TUẦN 2 · CHỦ ĐỀ 8

# Vai trò từng thành phần

| Control Plane | Vai trò |
|---|---|
| **kube-apiserver** | Cổng duy nhất vào cluster. Xác thực, phân quyền, ghi/đọc etcd. Mọi thành phần khác giao tiếp qua đây |
| **etcd** | CSDL key-value lưu toàn bộ trạng thái cluster. Mất etcd là mất cluster, nên phải backup |
| **kube-scheduler** | Chọn node cho Pod mới dựa trên tài nguyên còn trống, affinity, taint/toleration |
| **controller-manager** | Chạy các controller (Deployment, ReplicaSet, Node...), so sánh trạng thái thực với mong muốn rồi sửa |

| Worker Node | Vai trò |
|---|---|
| **kubelet** | Agent trên mỗi node. Nhận Pod được giao, gọi runtime chạy container, báo cáo trạng thái và health check |
| **container runtime** | Chạy container thật qua chuẩn CRI: containerd, CRI-O |
| **kube-proxy** | Cấu hình iptables/IPVS để traffic tới **Service** được chia đều về các Pod |
| **CNI plugin** | Cấp IP cho Pod, cho Pod ở các node khác nhau nói chuyện được: Calico, Flannel, Cilium |

**Đối tượng cơ bản: Pod** (đơn vị nhỏ nhất, 1+ container) · **Deployment** (quản lý số bản sao, rolling update) · **Service** (IP/DNS ổn định trước nhóm Pod) · **Namespace** (chia cluster theo team/môi trường) · **ConfigMap / Secret** (cấu hình và thông tin nhạy cảm)

TUẦN 2 · CHỦ ĐỀ 8

# Luồng xử lý khi triển khai một ứng dụng

1. **kubectl apply** gửi Deployment (replicas: 3) tới **kube-apiserver**; apiserver xác thực rồi lưu vào **etcd**
2. **Deployment controller** thấy Deployment mới → tạo ReplicaSet → ReplicaSet tạo **3 Pod** (chưa có node)
3. **kube-scheduler** thấy Pod chưa có node → chọn node phù hợp và ghi lại qua apiserver
4. **kubelet** trên node đó thấy Pod được giao → gọi **containerd** pull image và chạy container
5. kubelet báo trạng thái **Running** về apiserver; **kube-proxy** cập nhật routing nếu có Service trỏ tới Pod

**Cơ chế cốt lõi:** khai báo trạng thái mong muốn, hệ thống tự điều chỉnh. Pod chết thì controller tạo Pod mới thay thế.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: webapp }
spec:
  replicas: 3
  selector:
    matchLabels: { app: webapp }
  template:
    metadata:
      labels: { app: webapp }
    spec:
      containers:
      - name: web
        image: team/webapp:1.4.2
        resources:
          limits: { cpu: 500m, memory: 512Mi }
```

🎯 **Đầu ra:** Vẽ lại được kiến trúc Kubernetes không nhìn tài liệu

TỔNG KẾT

# Kết quả đạt được sau 2 tuần

| # | Mảng kiến thức | Đầu ra theo lộ trình | Trạng thái |
|---|---|---|---|
| 1 | Linux: CPU, RAM, disk, process | Kiểm tra và phân bổ được tài nguyên; giải thích được cho người khác | ✅ |
| 2 | Network: IP, routing, DNS, kết nối VM | Vẽ và giải thích được luồng kết nối mạng giữa các VM trong Lab | ✅ |
| 3 | Storage: partition, LVM, filesystem, mount | Tự thêm được disk mới, tạo LVM và mount vào hệ thống | ✅ |
| 4 | User, group và permission | Tạo và phân quyền được theo đúng yêu cầu của ứng dụng | ✅ |
| 5 | Service (systemd) và firewall | Quản lý được vòng đời dịch vụ; mở/đóng port theo yêu cầu | ✅ |
| 6 | Hypervisor và vòng đời VM | Thao tác đầy đủ vòng đời VM trên hypervisor đang dùng | ✅ |
| 7 | Container | Giải thích được cơ chế container và khác biệt so với VM trong 5 phút | ✅ |
| 8 | Kiến trúc Kubernetes | Vẽ lại được kiến trúc Kubernetes không nhìn tài liệu | ✅ |

**Điểm mạnh:** .....

**Cần củng cố thêm:** .....

# Cảm ơn anh/chị đã lắng nghe

Kế hoạch tiếp theo: .....

Hỏi & Đáp

Lê Tấn Lực · Đỗ Hoàng Hiếu